Enrollment No.: 240840116021

Practical 8

Aim: Create interactive visualizations using Plotly Express and Plotly Graph Objects.

Task 1: Import the required libraries and create the dataset

In this task, we import Pandas for handling tabular data and Plotly Express and Plotly Graph Objects for creating interactive visualizations.

We then create a small sample sales dataset containing monthly sales, products, regions, units sold, and profit. This dataset will be used in the following tasks.

In [1]:
```
# Import the libraries required for data handling and interactive visualization
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
```

In [2]:
```
# Create a sample dataset containing monthly sales information
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
    "Product": ["Laptop", "Mobile", "Tablet", "Laptop", "Mobile", "Tablet",
                "Laptop", "Mobile", "Tablet", "Laptop", "Mobile", "Tablet"],
    "Region": ["North", "South", "East", "West", "North", "South",
               "East", "West", "North", "South", "East", "West"],
    "Sales": [120000, 135000, 150000, 145000, 160000, 175000,
              180000, 170000, 190000, 210000, 230000, 250000],
    "Units_Sold": [12, 30, 25, 14, 35, 30, 18, 38, 32, 21, 45, 40],
    "Profit": [18000, 27000, 30000, 22000, 32000, 35000,
               28000, 34000, 38000, 33000, 46000, 50000]
}
```

In [3]:
```
# Convert the dictionary into a Pandas DataFrame
df = pd.DataFrame(data)

# Display the complete dataset
df
```

Out[3]:

	Month	Product	Region	Sales	Units_Sold	Profit
0	Jan	Laptop	North	120000	12	18000
1	Feb	Mobile	South	135000	30	27000
2	Mar	Tablet	East	150000	25	30000
3	Apr	Laptop	West	145000	14	22000
4	May	Mobile	North	160000	35	32000
5	Jun	Tablet	South	175000	30	35000
6	Jul	Laptop	East	180000	18	28000
7	Aug	Mobile	West	170000	38	34000
8	Sep	Tablet	North	190000	32	38000
9	Oct	Laptop	South	210000	21	33000
10	Nov	Mobile	East	230000	45	46000
11	Dec	Tablet	West	250000	40	50000

Task 2: Inspect the dataset

Before creating visualizations, we inspect the dataset to understand its structure.

- `head()` displays the first five records.
- `shape` returns the number of rows and columns.
- `info()` displays column names, data types, and non-null counts.

In [4]:
```
# display the first five records
df.head()
```

Out[4]:

	Month	Product	Region	Sales	Units_Sold	Profit
0	Jan	Laptop	North	120000	12	18000
1	Feb	Mobile	South	135000	30	27000
2	Mar	Tablet	East	150000	25	30000
3	Apr	Laptop	West	145000	14	22000
4	May	Mobile	North	160000	35	32000

In [5]:
```
# display the shape of the dataset
df.shape
```

Out[5]:
```
(12, 6)
```

In [6]:
```
# display the basic datatype and Null values related information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Month       12 non-null     object
 1   Product     12 non-null     object
 2   Region      12 non-null     object
 3   Sales       12 non-null     int64 
 4   Units_Sold  12 non-null     int64 
 5   Profit      12 non-null     int64 
dtypes: int64(3), object(3)
memory usage: 708.0+ bytes
```

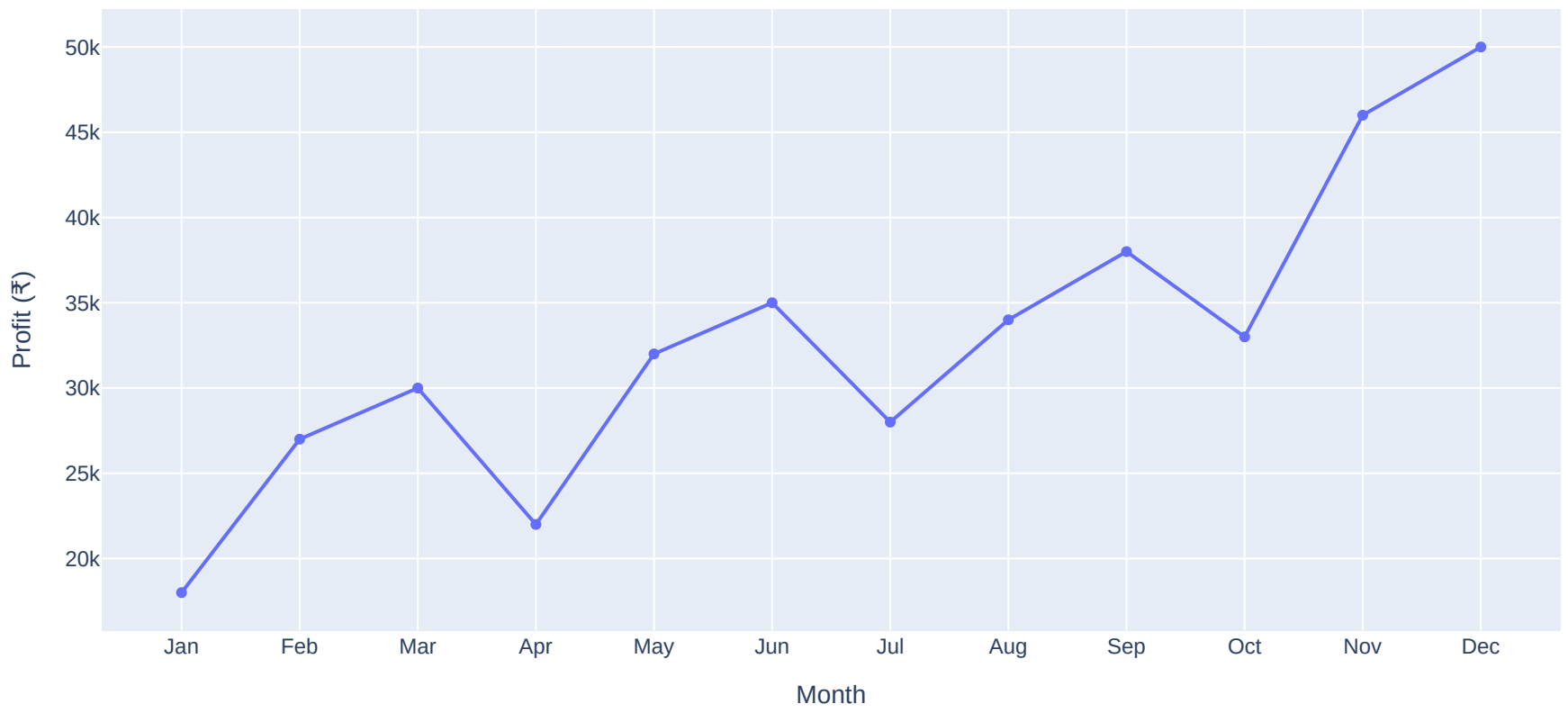
Task 3: Create an interactive line chart using Plotly Express

A line chart helps us understand how a value changes over time.

Here, we plot monthly profit. Plotly makes the chart interactive, so we can hover over data points and zoom into the chart.

In [7]:
```
# Create an interactive line chart to analyze monthly profit
fig = px.line(
    df,
    x="Month",
    y="Profit",
    title="Monthly Profit Analysis",
    markers=True,
    labels={"Month": "Month", "Profit": "Profit (₹)"}
)

# Display the interactive chart
fig.show()
```



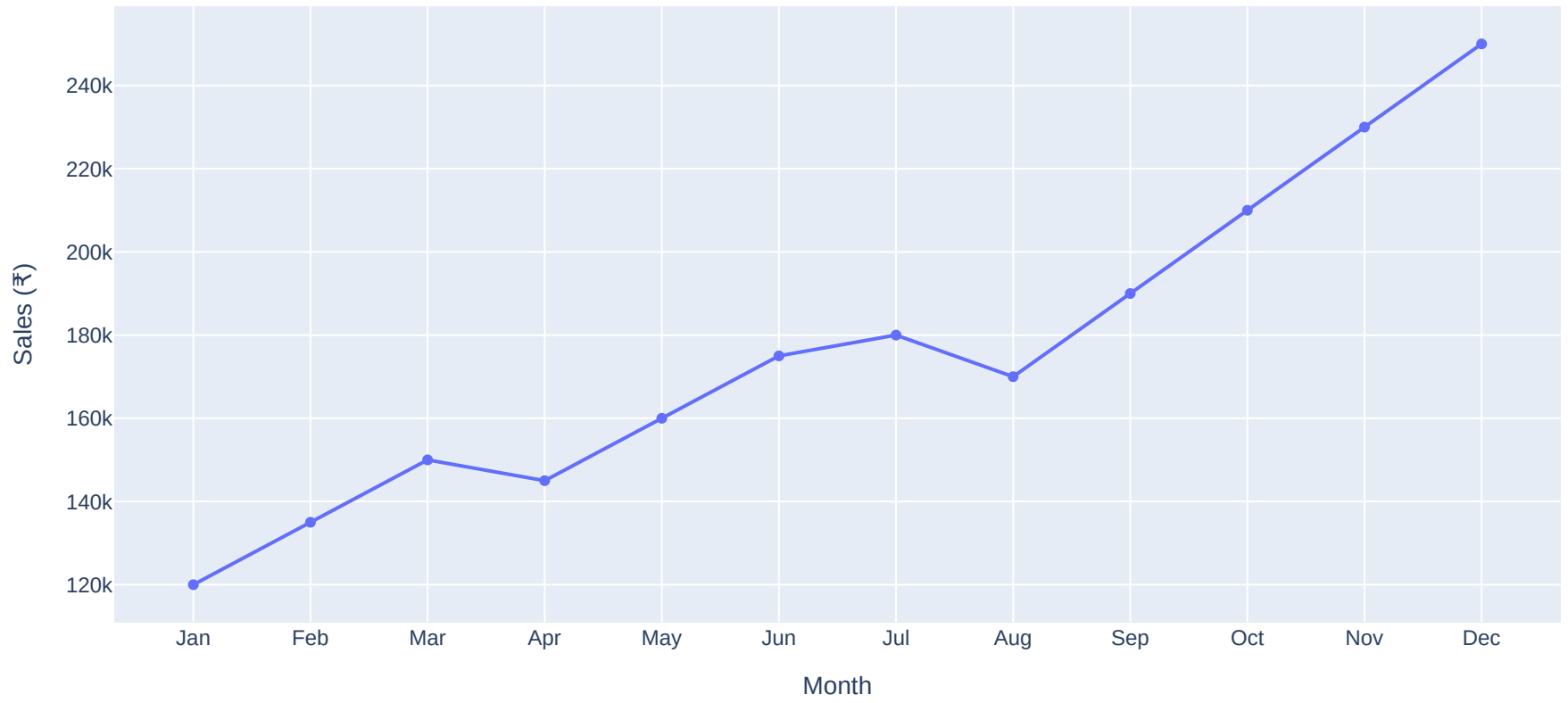
Task 4: Improve the hover information

Hover information provides additional details when we move the mouse pointer over a data point.

In this chart, we display sales for each month along with the related product, region, profit, and units sold.

In [8]:
```python
# Create a line chart with additional information on hover
fig = px.line(
    df,
    x="Month",
    y="Sales",
    title="Monthly Sales Analysis",
    markers=True,
    hover_data={
        "Product": True,
        "Region": True,
        "Profit": True,
        "Units_Sold": True
    },
    labels={"Sales": "Sales (₹)"}
)

# Display the chart with detailed hover information
fig.show()
```



Task 5: Create an interactive bar chart

A bar chart is useful for comparing values across different categories.

First, we group the dataset by product and calculate the total sales for each product. We use `reset_index()` to convert the grouped result into a DataFrame, which can be passed directly to Plotly Express.

In [9]:
```python
# Group the data by product and calculate total sales
product_based_grouped_data = (
    df.groupby("Product", as_index=False)["Sales"]
      .sum()
)

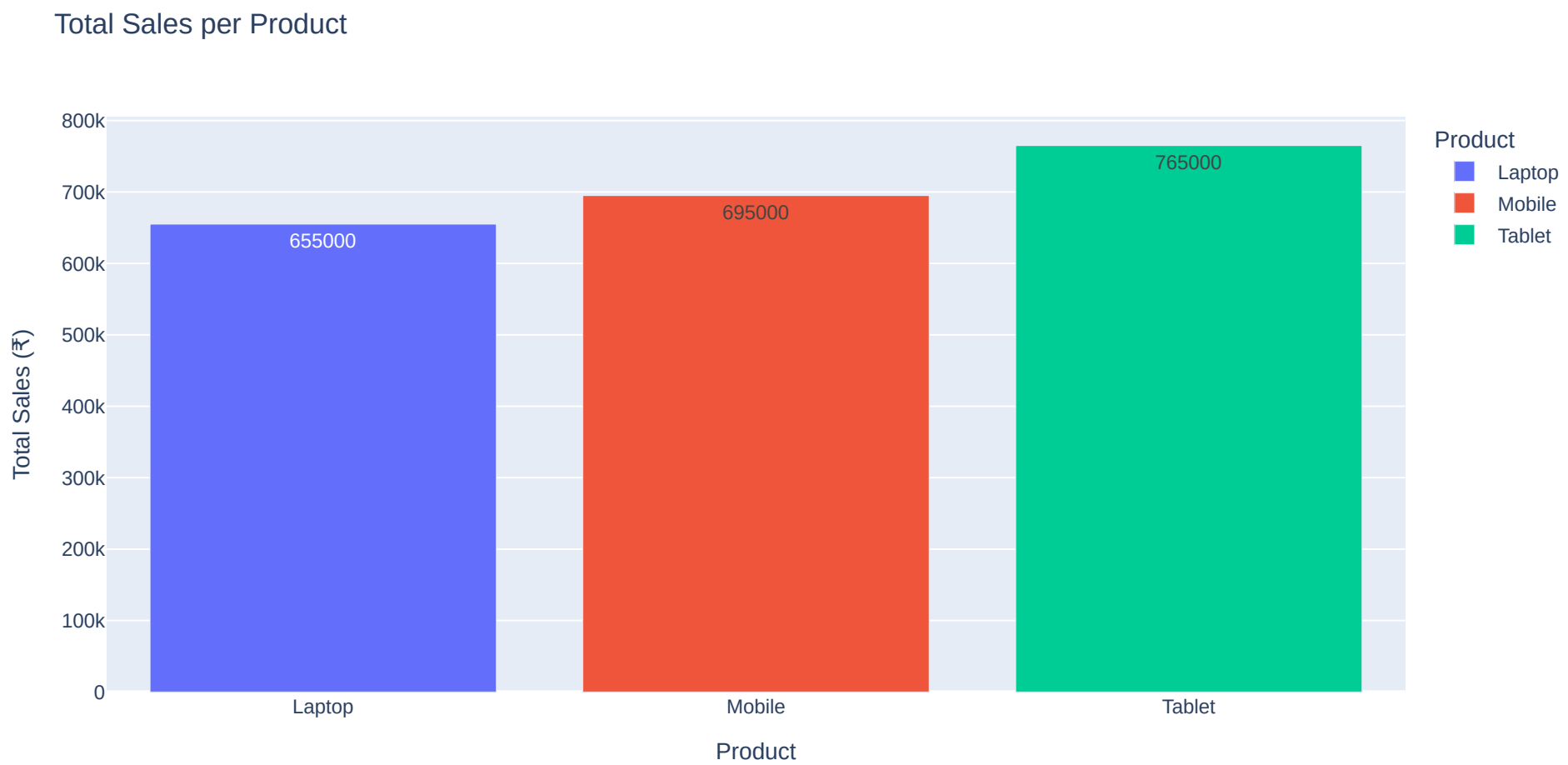
# Display the summarized sales data
product_based_grouped_data
```

Out[9]:

	Product	Sales
0	Laptop	655000
1	Mobile	695000
2	Tablet	765000

In [10]:
```
# Create an interactive bar chart to compare product-wise sales
fig = px.bar(
    product_based_grouped_data,
    x="Product",
    y="Sales",
    title="Total Sales per Product",
    color="Product",
    text="Sales",
    labels={"Sales": "Total Sales (₹)"}
)

# Display the interactive bar chart
fig.show()
```



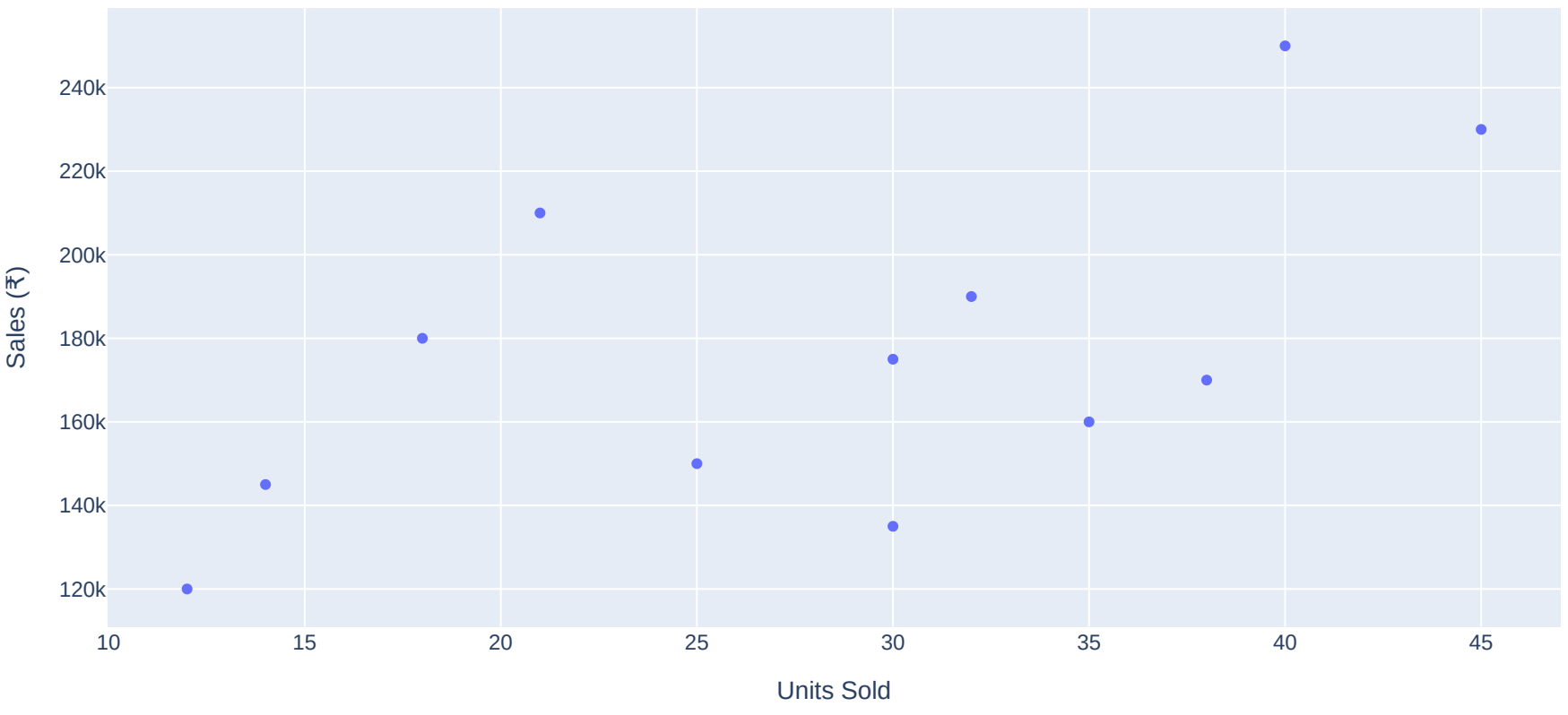
Task 6: Create an interactive scatter plot

A scatter plot helps us understand the relationship between two numerical variables.

Here, we compare the number of units sold with sales revenue. Each point represents one month's sales record.

In [11]:
```python
# Create a scatter plot to examine the relationship between units sold and sales
fig = px.scatter(
    df,
    x="Units_Sold",
    y="Sales",
    title="Relationship Between Units Sold and Sales",
    labels={
        "Units_Sold": "Units Sold",
        "Sales": "Sales (₹)"
    },
    hover_data=["Month", "Product", "Region"]
)

# Display the interactive scatter plot
fig.show()
```



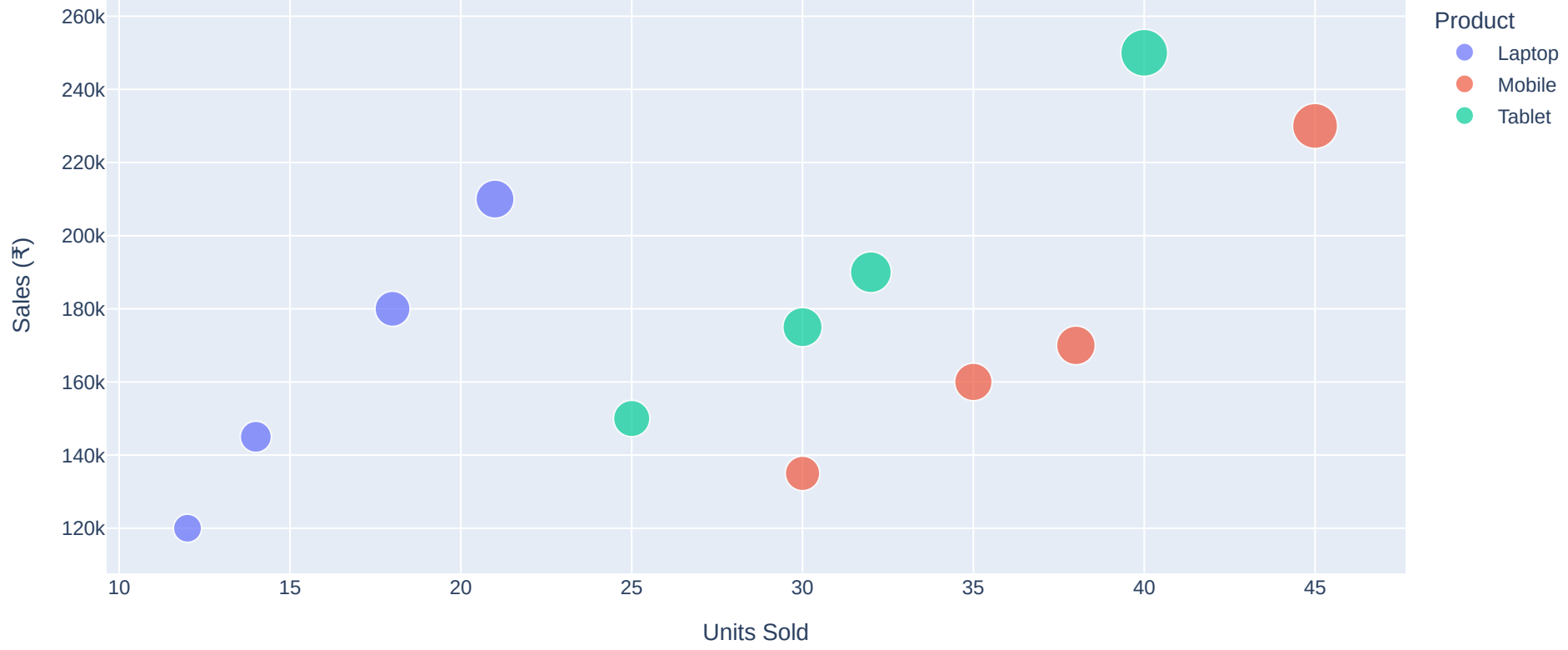
Task 7: Create an interactive scatter plot with color

In this task, we use the `color` parameter to distinguish products in the scatter plot.

Different colors make it easier to compare the sales and units sold for Laptop, Mobile, and Tablet products.

In [12]:
```
# Create a scatter plot with different colors for each product
fig = px.scatter(
    df,
    x="Units_Sold",
    y="Sales",
    color="Product",
    size="Profit",
    hover_name="Month",
    hover_data=["Region"],
    title="Sales and Units Sold by Product",
    labels={
        "Units_Sold": "Units Sold",
        "Sales": "Sales (₹)",
        "Profit": "Profit (₹)"
    }
)

# Display the interactive scatter plot
fig.show()
```



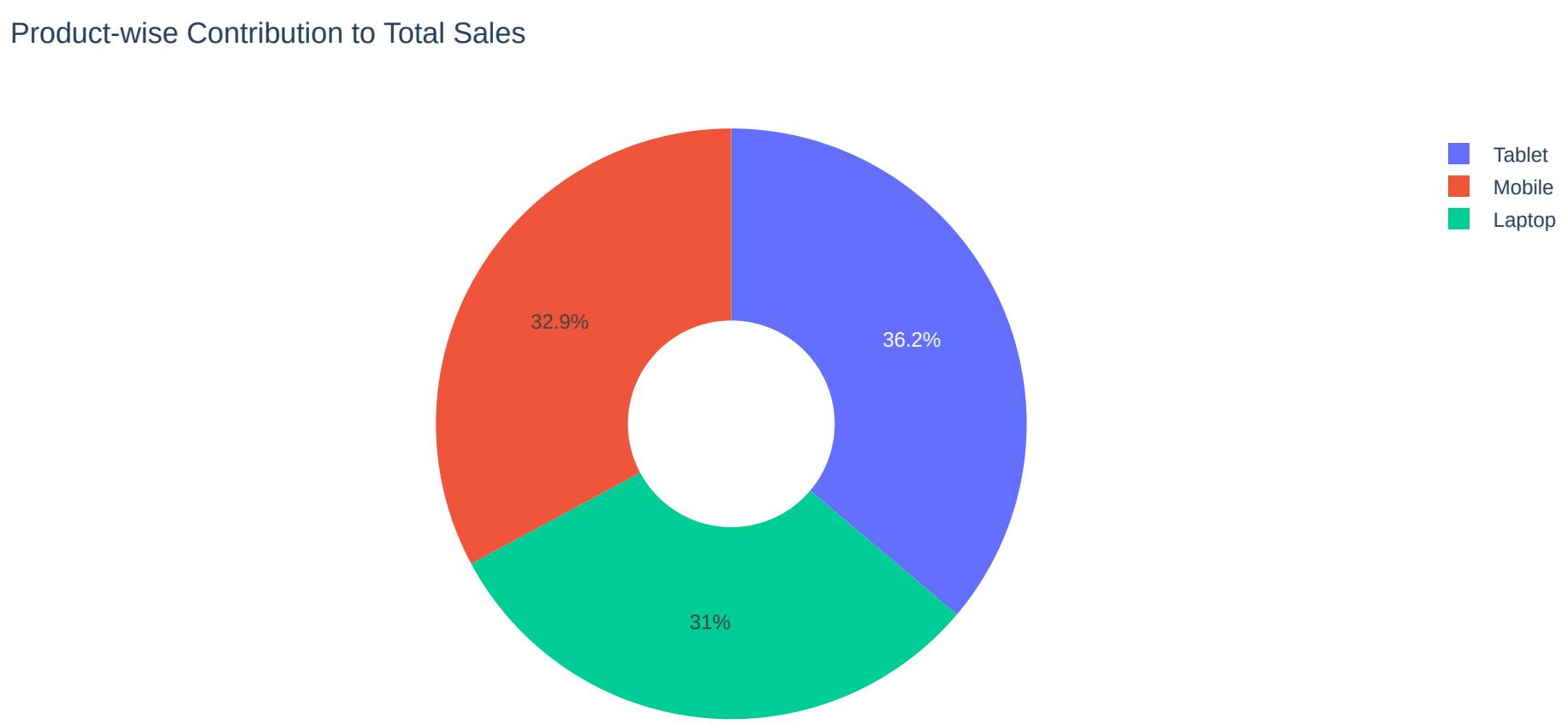
Task 8: Create an interactive pie chart using Plotly Express

A pie chart represents the contribution of each category to the total.

Here, we visualize the percentage contribution of each product to total sales. The chart displays product names, sales values, and percentages when we hover over the slices.

In [13]:
```
# Create a pie chart to show each product's share of total sales
fig = px.pie(
    product_based_grouped_data,
    names="Product",
    values="Sales",
    title="Product-wise Contribution to Total Sales",
    hole=0.35
)

# Display the interactive pie chart
fig.show()
```



Task 9: Create an interactive bar chart using Plotly Graph Objects

Plotly Graph Objects provides more control over individual chart elements.

In this task, we create a bar chart using `go.Figure()` and `go.Bar()`. We add a chart title, axis labels, bar colors, and hover information to make the visualization informative.

In [14]:
```
# Create a bar chart using Plotly Graph Objects
fig = go.Figure(
    data=[
        go.Bar(
            x=product_based_grouped_data["Product"],
            y=product_based_grouped_data["Sales"],
            marker=dict(
                color=["#636EFA", "#EF553B", "#00CC96"]
            ),
            text=product_based_grouped_data["Sales"],
            textposition="auto",
            hovertemplate=(
                "Product: %{x}<br>"
                "Total Sales: ₹%{y:,.0f}"
                "<extra></extra>"
            )
        )
    ]
)

# Customize the chart layout
fig.update_layout(
    title="Product-wise Sales Using Plotly Graph Objects",
    xaxis_title="Product",
    yaxis_title="Total Sales (₹)",
    template="plotly_white"
)

# Display the interactive chart
fig.show()
```

